Basic MQL and Document Modeling

A support application must find and change the intended ticket.

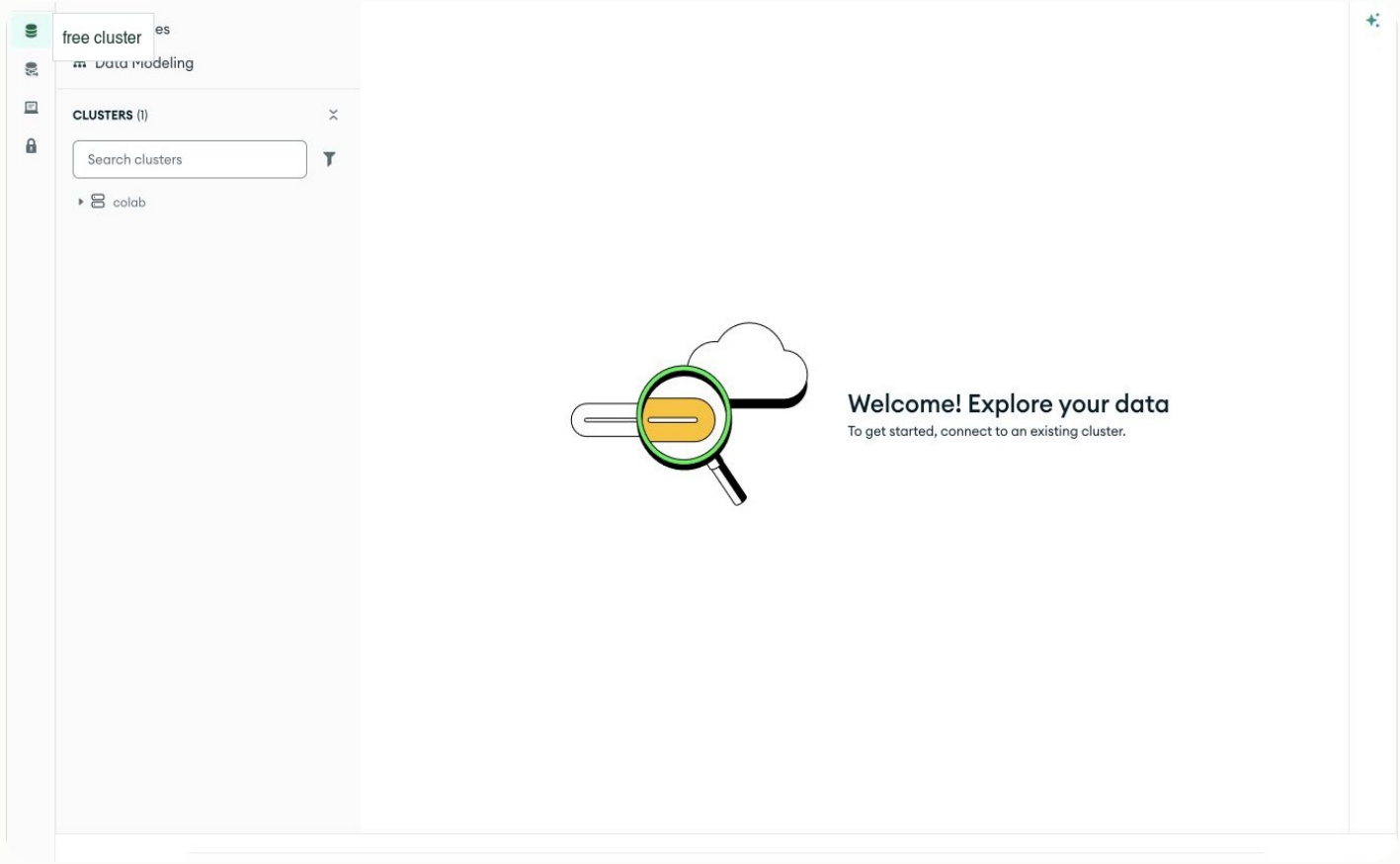
Day 1: Python queries with observable results
Day 2: document boundaries for a ticket page

SQL concepts and Python MongoDB operations

Purpose	SQL idea	Python / PyMongo
Choose a source	FROM tickets	database["tickets"]
Choose records	WHERE status = 'open'	{"status": "open"}
Choose output fields	SELECT ticket_id	{"ticket_id": 1, "_id": 0}
Order and restrict	ORDER BY ticket_id LIMIT 2	.sort("ticket_id", 1).limit(2)

The concepts transfer. Null, arrays, and BSON types need their own rules.
A filter is a dictionary, not a SQL string.

Atlas deployment and database connection



Project: cloud administration

Cluster: database service

Database: collections

Collection: documents

This capture shows a deployment while Data Explorer requests a connection.

Interface captured August 25, 2026. Names redacted. This is not a successful query result.

Colab connects from a remote runtime

Notebook action	What it establishes
Show runtime IP and practice database name	Where the code runs and where this lesson writes
Add the printed IP as a temporary /32 rule	Permit that IPv4 source to attempt a connection
Use a database user and hidden URI prompt	Authenticate separately from the Atlas website login
Send ping, then insert the teaching data	Test reachability, then the required data permissions

Today, Colab connects to your Atlas cluster. Keep TLS verification enabled.
If connection fails, use Setup C's diagnostic category to choose the next check.

MongoDB University browser activities on slide 37

JSON, Python, and shell notation

Value or operation	Notebook Python	mongosh
Null value	None	null
Boolean value	True / False	true / false
Find one document	tickets.find_one({...})	db.tickets.findOne({...})
Update-result property	result.modified_count	result.modifiedCount
Typed date	datetime(..., tzinfo=timezone.utc)	ISODate("2026-02-01T23:10:00Z")

The slides use Python. Chapter 10 labels its runnable shell examples.
MongoDB stores BSON, including typed dates that JSON strings do not create.

The notebook has six teaching tickets

Ticket	Category	Priority	Status
1001	streetlight	high	open
1002	sanitation	medium	in_progress
1003	water	urgent	resolved
1004	parks	low	new
1005	sanitation	high	resolved
1006	streetlight	medium	in_progress

Six supplied tickets; separate test ticket 1099 is added later.
Next: complete fields, types, and relationships before querying.

Ticket fields and value types

One document describes one ticket. These are the supplied fields, not an installed schema validator.

Field	Type	Meaning and example values
_id	ObjectId	MongoDB document identity, generated on insert; separate from ticket_id
ticket_id	integer	Case ticket number: 1001–1006; disposable test ticket: 1099
category	string	Service area: streetlight, sanitation, water, or parks in this fixture
priority	string	Importance: low, medium, high, or urgent; independent of status
status	string	Workflow state: new, open, in_progress, or resolved in this fixture
subject	string	Ticket description, such as Streetlight dark near bus stop
opened_at	BSON date	Opening instant; 1001 opens at 2026-02-01 23:10 UTC

Python datetime creates typed dates here. Extended JSON uses a $date wrapper to display them.
A plain quoted date string remains text. ticket_id has a unique index in the practice collection.

Related people and arrays inside a ticket

Day 1 uses one tickets collection. Requester details and events live inside each ticket document.

Field or path	Type	Meaning and example values
requester	object	Embedded requester snapshot containing user_id and display_name
requester.user_id	integer	Case user identity: 101–104; user 101 requested tickets 1001 and 1005
requester.display_name	string	Readable label, such as Requester 101; copied into the ticket
assignee_id	integer or null	Assigned staff identity: 201 or 202; ticket 1004 has None / BSON null
tags	array of strings	Labels such as lighting and safety; query whether the array contains a value
events	array of objects	Supplied event records; tickets 1005 and 1006 have empty arrays here

user_id and assignee_id are logical IDs; Day 1 has no separate users collection or enforced foreign key.
Empty events arrays omit history in this fixture. Test 1099 initially has no tags or assignee_id field.

Event fields and practice markers

Each events element has the four fields below. The final two markers belong to the ticket itself.

Field or path	Type	Meaning and example values
events.event_id	integer	Event identity: 5001–5008 in the fixture; the write exercise appends 5999
events.type	string	Event kind: created, assigned, or status_changed in this fixture
events.actor_role	string	Role of the actor: resident or agent; this fixture omits actor_id
events.at	BSON date	When the event occurred; 5001 is at 2026-02-01 23:10 UTC
course_fixture	string	cst4714 marks all six supplied tickets and the disposable test ticket
test_record	boolean if present	True only on test ticket 1099; absent from the six supplied tickets

The CSV case's event_type becomes type, and event_at becomes at. This is a selected teaching model.
An event_id inside an array is not automatically unique. The append exercise supplies its own guard.

A filter and a projection

```
filter_doc = {
    "status": {"$in": ["new", "open", "in_progress"]},
    "priority": {"$in": ["high", "urgent"]}
}
projection = {"_id": 0, "ticket_id": 1, "status": 1}
for document in tickets.find(filter_doc, projection):
    print(document)
```

```
Result: {'ticket_id': 1001, 'status': 'open'}
```

Both conditions must hold. Ticket 1003 is urgent but resolved.
Projection changes the returned fields, not the stored document.

A cursor, ordering, and a limit

```
cursor = tickets.find(
    {"status": {"$in": ["new", "open", "in_progress"]}},
    {"_id": 0, "ticket_id": 1, "opened_at": 1}
).sort([("opened_at", -1), ("ticket_id", 1)]).limit(2)

for document in cursor:
    print(document)
```

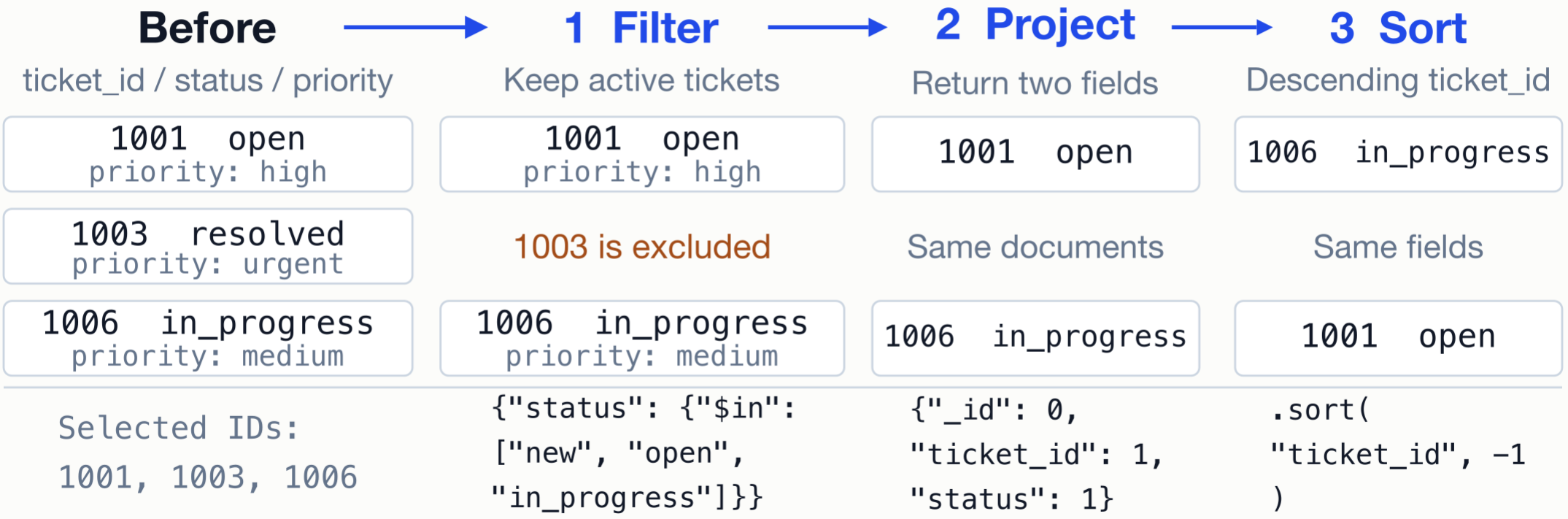
Returned IDs: 1006, then 1004.
Newest opening first; ticket_id gives a tie-break order.

Iteration reads the cursor. To repeat a consumed query, call find again.

Filtering, projection, and ordering

Filter chooses documents. Projection chooses fields. Sort orders results.

Three selected tickets from the six-ticket fixture; other fields are omitted here.



These are conceptual steps. The stored documents are unchanged.

Continue to nested fields: slide 13

Nested fields and simple arrays

```
for document in tickets.find(
    {"requester.user_id": 101}, {"_id": 0, "ticket_id": 1}
):
    print(document)

for document in tickets.find(
    {"tags": "safety"}, {"_id": 0, "ticket_id": 1}
):
    print(document)
```

Both queries return IDs 1001 and 1005 for different reasons.
The dotted name reaches a nested field. The tag condition matches an array value.

Two conditions can match different events

Ticket 1001 event	type	actor_role
5001	created	resident
5002	assigned	agent

```
wrong_question = {
    "events.type": "created",
    "events.actor_role": "agent"
}
```

Ticket 1001 matches this filter, but no single event says an agent created it.

Static diagram on slide 16

$elemMatch requires one qualifying event

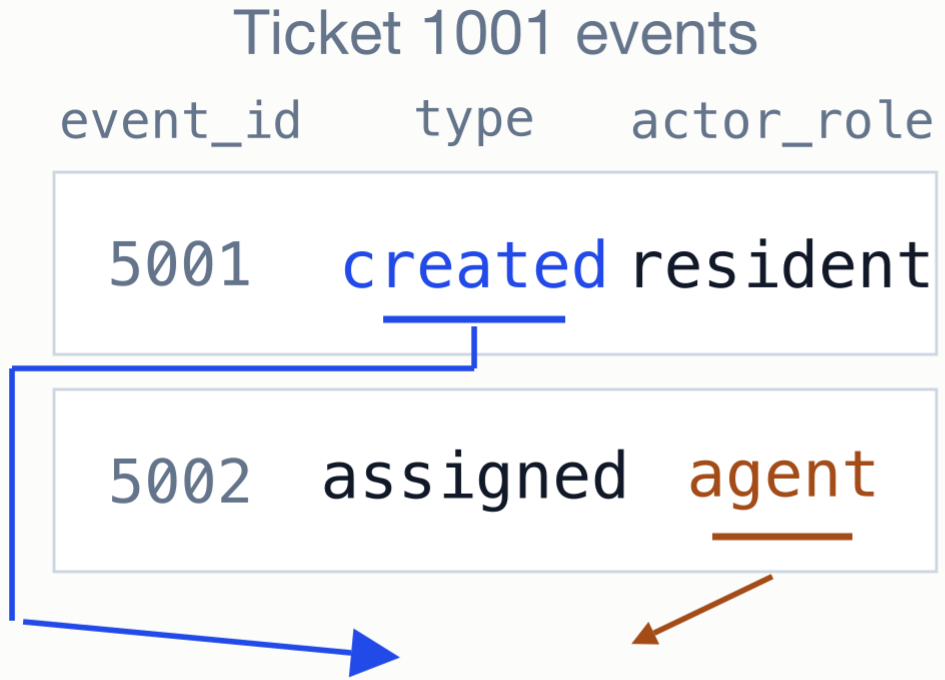
```
agent_created = {
    "events": {
        "$elemMatch": {"type": "created", "actor_role": "agent"}
    }
}
print(list(tickets.find(agent_created, {"_id": 0, "ticket_id": 1})))
```

Result: []
The empty result correctly answers the question for this fixture.

Changing type to status_changed returns tickets 1002 and 1003.
A matching document can still contain other, nonmatching events.

Static diagram on slide 16

Array conditions and one matching event



Separate dotted conditions

Both values occur somewhere
Ticket matches



`$elemMatch`

No event contains both values
Ticket does not match

Continue to slide 17

One disposable ticket for write practice

```
test_filter = {
    "ticket_id": 1099, "test_record": True, "course_fixture": "cst4714"
}
tickets.delete_many(test_filter)
tickets.insert_one({
    **test_filter, "status": "new", "subject": "Disposable MQL test",
    "events": []
})
```

This reset only targets marked test ticket 1099.
The notebook also creates a unique index on ticket_id.

Run after the connection and six-ticket setup. No sample database is changed.

A repeated $set can match without changing

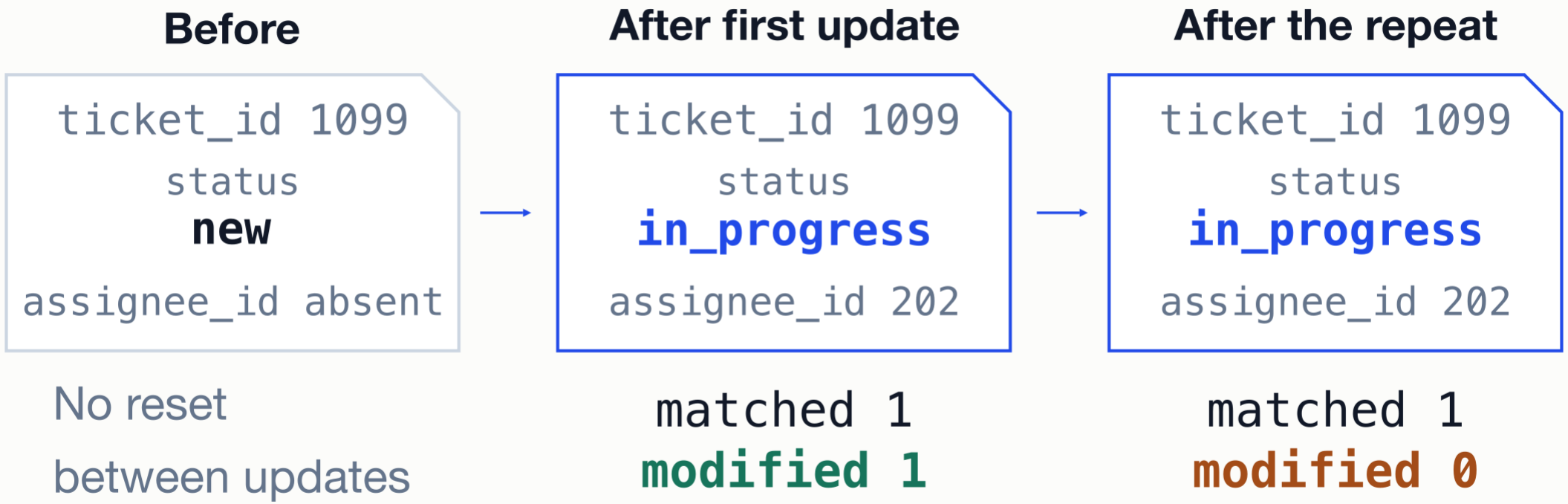
```
result = tickets.update_one(
    test_filter,
    {"$set": {"status": "in_progress", "assignee_id": 202}}
)
print(result.matched_count, result.modified_count)
print(tickets.find_one(test_filter, {"_id": 0, "status": 1}))
```

Execution	Matched	Modified	Stored status
First	1	1	in_progress
Same update again	1	0	in_progress

Static diagram on slide 19

The first update and the repeat

```
$set  status = "in_progress", assignee_id = 202
```



The second filter still matches. The requested values are already stored.

Continue to slide 20

An expected-state condition can prevent a write

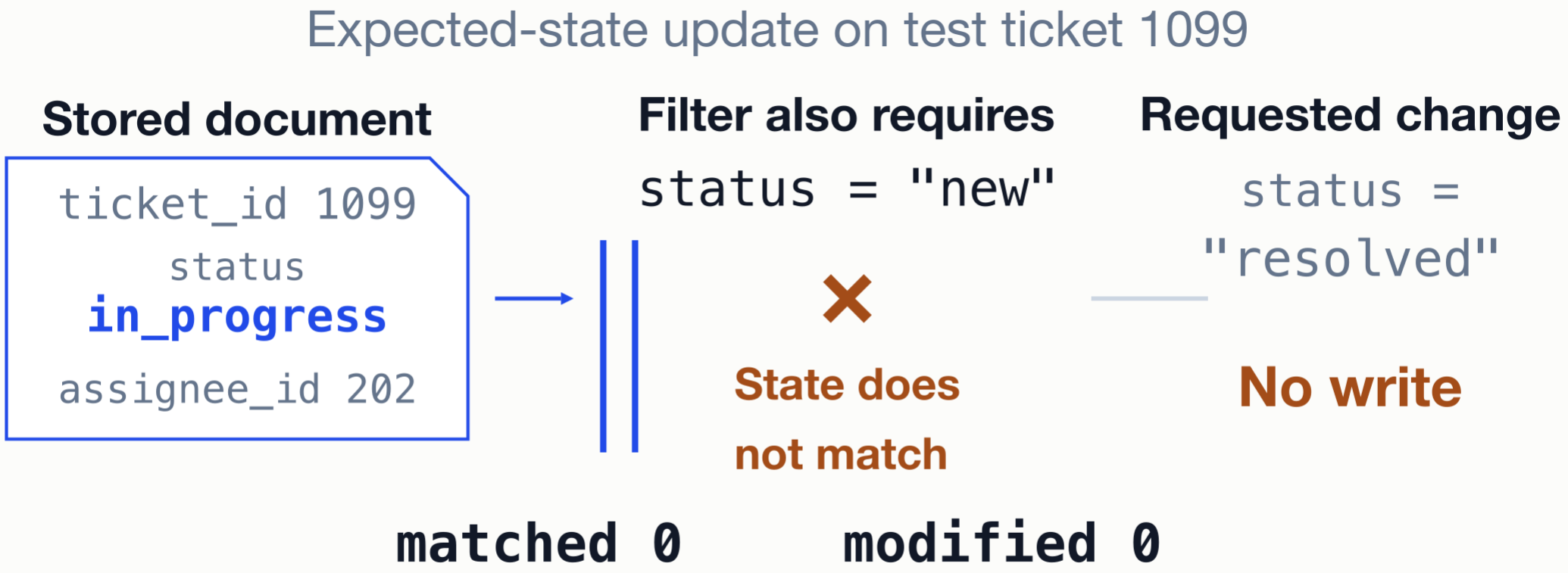
```
result = tickets.update_one(
    {**test_filter, "status": "new"},
    {"$set": {"status": "resolved"}}
)
print(result.matched_count, result.modified_count)
print(tickets.find_one(test_filter, {"_id": 0, "status": 1}))
```

Result: 0 0. Status remains in_progress.
The current document no longer satisfies status = new.

A zero match is an application outcome to interpret, not proof of a connection failure.

Static diagram on slide 21

The expected state in an update filter



Status stays in_progress. Reload the document before deciding what to do next.

Continue to slide 22

Appending an event and repeating the request

```
result = tickets.update_one(
    {**test_filter, "events.event_id": {"$ne": 5999}},
    {"$push": {"events": {
        "event_id": 5999, "type": "status_changed",
        "actor_role": "agent"
    }}}
)
print(result.matched_count, result.modified_count)
```

First request: 1 1. Repeated request: 0 0.
The filter refuses a second append of event 5999.

This append and the earlier status update were separate writes.
Day 2 combines related changes in a single update.

Deletion with an exact target

```
print(tickets.find_one(test_filter, {"_id": 0, "ticket_id": 1}))
result = tickets.delete_one(test_filter)
print("Deleted:", result.deleted_count)
print("Remaining test record:", tickets.find_one(test_filter))
print("Other tickets:", tickets.count_documents(
    {"course_fixture": "cst4714"}
))
```

Deleted: 1. Remaining test record: None. Other tickets: 6.
Repeating the same delete reports 0.

An empty filter matches everything. Preview and deletion are separate operations, so a preview alone is not a lock against concurrent changes.

Lab 1: the lighting and sanitation queue

Show active streetlight or sanitation tickets, newest first.
Include priority so the crews can plan their work.
Explain why high-priority ticket 1005 stays out of this queue.

Then compare the array queries and repeat the $set update.
Read the changed test ticket before deleting it.

Individual work. Submit the completed notebook with one short explanation.
Run cleanup before finishing. No extra report or Atlas screenshot.

From a new account to a stored document

How can we tell whether MongoDB saved the account we just opened?

First, create an account and read it back in the MongoDB shell.
Then apply the same idea to a page that shows two recent events.
The lab gives you a collection of your own to query.

Create an account with insertOne

Our accounts collection starts empty. Each account will be one document.

MongoDB shell command

```
const saved = db.accounts.insertOne({
  owner: "Ari", status: "open"
})
saved
```

What the shell returns

```
{
  acknowledged: true,
  insertedId: ObjectId("6aed03ee82c481486b6c4e15")
}
```

Which value would let us find this exact account again?

Read the account and insert a batch

Use the ID from the write to look up Ari’s account in the same collection.

Read the stored document in mongosh

```
db.accounts.findOne({ _id: saved.insertedId })
// Result
{ _id: ObjectId("6aed03ee82c481486b6c4e15"),
  owner: "Ari", status: "open" }
```

Add two accounts in one call

```
db.accounts.insertMany([
  { owner: "Bo", status: "open" },
  { owner: "Cam", status: "open" }
])
```

We started with Ari. How many documents will there be after this call runs once?

Status and history in one document update

```
result = demo_tickets.update_one(
    {"ticket_id": 1001, "status": "open"},
    {
        "$set": {"status": "in_progress"},
        "$push": {"events": {
            "event_id": 5099, "type": "status_changed"
        }}
    }
)
print(result.matched_count, result.modified_count)
```

The status change and event append form one atomic document write. Repeating it finds no open ticket 1001: 0 0, without a second event.

This worked example starts with an open ticket. Separate documents need coordination.

Current contact details and historical snapshots

Stored fact	Owner	Effect of a contact correction
requester_id: 101	Ticket stores the relationship	ID stays the same
Current email for user 101	Authoritative user document	Update one user record
requester_name_at_open	Ticket owns a historical snapshot	Keep it only if the historical meaning is intended

The page can read the user by ID for current contact details.
A copied current email would introduce another value to maintain.

A reference on each event allows independent growth

```
{"ticket_id": 1001, "requester_id": 101, "status": "open"}
```

```
{
  "event_id": 5002,
  "ticket_id": 1001,
  "event_type": "assigned",
  "event_at": "2026-02-02T14:05:00Z"
}
```

Each event points to its ticket. The parent does not need an ever-growing ID list.
The timestamp above is JSON text; the runnable demo stores a BSON date.

Reading the ticket page from referenced data

```
ticket = page_tickets.find_one({"ticket_id": 1001}, {"_id": 0})
person = people.find_one({"user_id": ticket["requester_id"]}, {"_id": 0})
recent = list(history.find(
    {"ticket_id": ticket["ticket_id"]}, {"_id": 0}
).sort([("event_at", -1), ("event_id", -1)]).limit(2))
print(ticket["status"], person["display_name"])
print([event["event_id"] for event in recent])
```

Chapter example: open, Requester 101, event IDs [5002, 5001].
Three reads assemble the page. A separate document retains each event.

An event index starting with ticket_id and then the ordering fields can support this read.

Growing history and a bounded preview

Representation	What grows with each new event	Page-read consequence
All events embedded	The ticket document	One read, increasing document size
All events separate	The number of event documents	Ticket plus bounded history query
Separate history and a recent copy	History, but recent copy stays capped	Convenient preview with refresh work

A BSON document has a 16 MiB size limit.
A design can become awkward well before that limit.

A capped copy needs an explicit refresh and stale-data policy.

A model decision has an observable consequence

A separate user record lets a contact correction happen once.
Separate event records keep complete history without enlarging the ticket.

The page must retrieve the related records. If it needs one consistent version of all of them, the application needs stronger coordination.

Workloads matter in PostgreSQL too. PostgreSQL supports arrays and JSON.
Neither a SQL table nor a MongoDB collection chooses a good model for us.

Week 10 Lab 2

Open the Week 10 Lab 2 handout and its MongoDB University course link.
Follow the walkthrough to the first lab.

Part A. Lab log
Record a command and what it returned. Change a filter.
Which documents matched before and after the change?

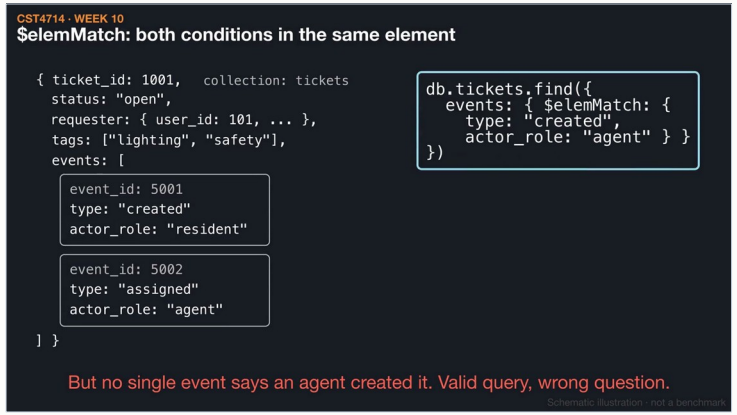
Part B. Read Chapter 10 and write about ticket 1003.
Open the reading prompt. Submit both parts in one Brightspace entry.

What did the query show?

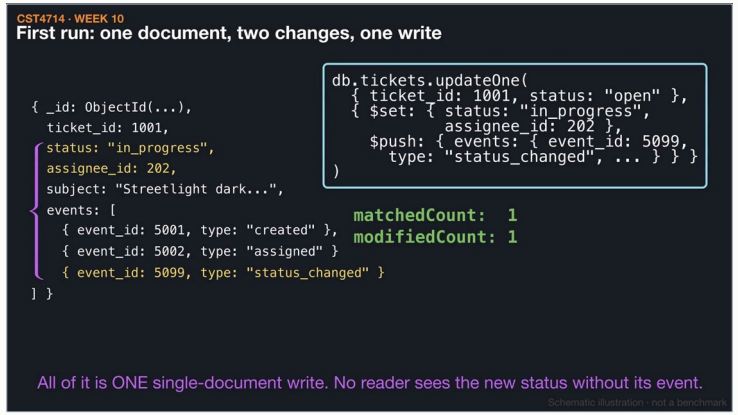
Name the collection and a field you saw in the result. Show how one change to a filter affected which documents matched. Use that observation and a Chapter 10 section to explain how a page can retrieve two recent events while retaining the complete history.

Finish the lab log and reading response in Brightspace.
Next week we aggregate results and enforce document rules.

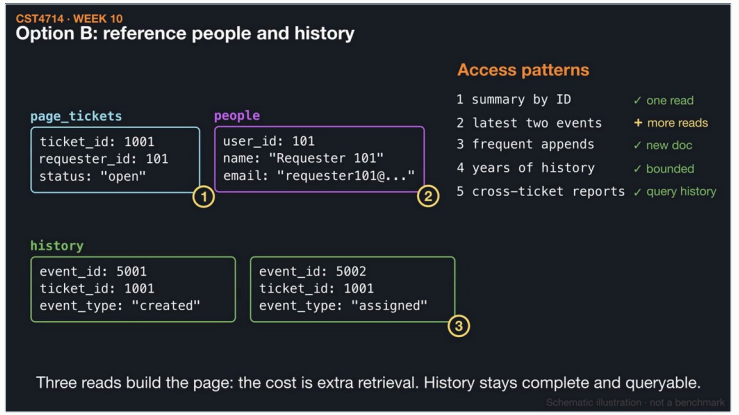
This week's videos



▶ Querying inside documents: arrays and $elemMatch (1:12)



▶ Did the write do what you meant? (1:06)



▶ Embed or reference? Let the workload decide (1:21)

In PowerPoint Slide Show, click a thumbnail to play full screen. Embedded videos work offline.

MongoDB University and Colab

The Week 10 Lab 2 handout links to the class activities and the Day 2 notebook.

	MongoDB University	Our Colab notebook
Workspace	Tools provided by the activity	Python cells in a notebook you save
Guidance	Follow the prompts when a workspace is available	Edit the course cells and explain the results
Database	Use the environment specified by the lesson	Day 2 uses local practice data

Day 2: Open the Week 10 Lab 2 handout

Optional orientation: Introduction to the IDE

Optional practice: Atlas Data Explorer

Return to slide 4

Relationship cardinality and stored shape

Relationship in this application	Cardinality	A modeling consequence
Ticket to requester	Many tickets to one user	A shared user can change independently
Ticket to events	One ticket to many events	History can outgrow an embedded array
Publisher to headquarters	One to one, by assumption	A small owned address can fit in its publisher

Cardinality describes how entities relate. It does not choose the storage by itself. Ownership, reads, writes, and growth still matter.

One publisher and its headquarters

A one-to-one example
One publisher owns one small address.

```
{
  "publisher_id": "p7",
  "name": "Example Press",
  "headquarters": {"city": "New York", "country": "US"}
}
```

Embedding returns this address with its publisher in one read.
An independently maintained address may warrant its own document.